# Lecture Supervision on a Circular Timetable: Efficient Open Domination of a Circular-Arc Graph

[Your name] ([Roll number])
Design and Analysis of Algorithms – Course Project

October 27, 2026

**Abstract**

$n$ lectures repeat every day on a circular timetable of $L = 1440$ minutes. We must choose a set $S$ of lectures such that every lecture overlaps exactly one lecture of $S$. This is an *efficient open dominating set* (a total perfect code) of the circular-arc overlap graph. We prove that every solution is a union of disjoint *blocks*, show that there are at most $2n$ candidate blocks, and give an $O(n^3)$ algorithm that anchors one block, unrolls the circle and runs a dynamic programme over the remaining blocks. The algorithm is implemented in Python, verified by an independent checker, and cross-checked against exhaustive search on more than a thousand random, planted and mutated instances. All code is available at `https://github.com/Revanth422/DAA`.

## Contents

**1 Problem statement and classification** **1**

**2 Structure of every solution** **2**

**3 The algorithm** **2**

**4 Correctness** **4**

**5 Complexity** **5**

**6 Implementation and reproducibility** **5**

**7 Experiments** **6**
7.1 Built-in tests and cross-checks . . . . . 6
7.2 Running time . . . . . 7

**8 Conclusion and limitations** **7**

**A Raw program output** **7**

**B Source code** **8**

## 1 Problem statement and classification

Time is divided into $L = 1440$ unit *cells* (minutes) arranged on a circle, so cell $L-1$ is followed by cell 0. Lecture $i$ ($1 \le i \le n$) is given by a start $s_i$ and an end $e_i$ (both in $\{0, \dots, L-1\}$) and occupies the arc

$$A_i = \{s_i, s_i+1, \dots, s_i+d_i-1\} \pmod{L}, \qquad d_i = \begin{cases} (e_i - s_i) \bmod L & e_i \neq s_i, \\ L & e_i = s_i, \end{cases}$$

so a lecture may wrap past midnight, and $s_i = e_i$ denotes a full-day lecture. Two distinct lectures *overlap* if $A_i \cap A_j \neq \emptyset$, i.e. if they share positive length; lectures that only touch at an endpoint do *not* overlap. Let $G$ be the overlap graph and $N(v)$ the *open* neighbourhood of $v$ (a lecture is never its own neighbour).

**Task.** Find $S \subseteq \{1,\dots,n\}$ such that $|N(v) \cap S| = 1$ for every lecture $v$, or report that none exists. Equivalently, the open neighbourhoods $N(s)$, $s \in S$, partition the vertex set. Such an $S$ is called an *efficient open dominating set* or *total perfect code* [2, 3].

**Classification.** $G$ is a circular-arc graph [4]. The problem is an exact-cover problem, and a brute-force search over all subsets needs $\Theta(2^n)$ candidates. We exploit the circular structure: a structural characterisation (Section 2) reduces the search to chains of "blocks" around the circle, which a dynamic programme over a linearised ("anchored") circle decides in polynomial time, using bit-set graph operations (technique: structural decomposition + dynamic programming [1]).

**Conventions.** Lectures are numbered $1,\dots,n$ in this report and in the program output (the source code uses $0,\dots,n-1$ internally). The complete input format is described in Section 6.

## 2 Structure of every solution

**Definition 1** (Block, region, hit)**.** A *block* is a pair $\{a,b\}$ of overlapping lectures with $N(a) \cap N(b) = \emptyset$. Its *region* is $R = A_a \cup A_b$. A lecture *w hits* a region $R$ if $A_w \cap R \neq \emptyset$ (a lecture of the block hits its own region).

Because $a \in N(b)$ and $b \in N(a)$ but $a \notin N(a)$, the condition $N(a) \cap N(b) = \emptyset$ says exactly that *no third lecture overlaps both a and b*. Consequently, if $w \notin \{a,b\}$ hits the region of a block $\{a,b\}$, then $w$ overlaps *exactly one* of $a,b$. $(*)$

**Lemma 1** (Decomposition)**.** *A nonempty set S is valid (every lecture overlaps exactly one lecture of S) if and only if $S = B_1 \cup \cdots \cup B_k$ ($k \geq 1$) for pairwise disjoint blocks $B_j$ with regions $R_j$ such that*

$$\textit{every lecture hits exactly one of the regions } R_1,\dots,R_k. \qquad (\star)$$

*In that case the regions $R_1,\dots,R_k$ are pairwise disjoint.*

*Proof.* ($\Rightarrow$) Every $v \in S$ has exactly one neighbour $p(v) \in S$, and $p(p(v)) = v$ because $v$ is a neighbour of $p(v)$ in $S$. Hence $S$ splits into the pairs $\{v, p(v)\}$; call them $B_1,\dots,B_k$. If some $w$ were in $N(a) \cap N(b)$ for $B_j = \{a,b\}$, then $w$ would have two neighbours in $S$; so each $B_j$ is a block. Now let $w$ be any lecture. If $w \notin S$, then $w$ hits $R_j$ iff $w$ overlaps a lecture of $B_j$; since $w$ has exactly one neighbour in $S$, it hits exactly one region. If $w \in B_i$, it hits $R_i$; if it also hit $R_j$, $j \neq i$, it would overlap some $x \in B_j$ in addition to $p(w)$, a contradiction. This proves $(\star)$. Finally, a cell lying in $R_i \cap R_j$ ($i \neq j$) would belong to some $x \in B_i$ and some $y \in B_j$, so $x$ would hit $R_j$ and $R_i$, contradicting $(\star)$.

($\Leftarrow$) Let $w$ be a lecture and $R_j$ the unique region it hits. Every neighbour $x \in S$ of $w$ lies in some $B_i$, and then $w$ hits $R_i$, so $i = j$: all neighbours of $w$ in $S$ lie in $B_j = \{a,b\}$. If $w \notin B_j$, then $w$ hits $R_j$, so it overlaps at least one of $a,b$, and by $(*)$ exactly one. If $w = a$, its only possible neighbour in $S$ is $b$, and $a$ overlaps $b$. In all cases $w$ has exactly one neighbour in $S$. □

**Corollary 1** (Whole-circle block)**.** *If $\{a,b\}$ is a block with $A_a \cup A_b = \mathbb{Z}_L$, then $S = \{a,b\}$ is valid.*

*Proof.* Every lecture has at least one cell, so it hits the single region $\mathbb{Z}_L$ exactly once; apply Lemma 1. □

**Lemma 2** (Few blocks)**.** *There are at most $2n$ blocks whose region is not the whole circle.*

*Proof.* Let $\{a,b\}$ be a block with $R \neq \mathbb{Z}_L$. Cut the circle at a cell outside $R$; then $A_a$ and $A_b$ become ordinary intervals. They overlap, so exactly one of two cases occurs.

*Nested.* One arc contains the other, say $A_b \subseteq A_a$ (if the arcs are equal, let $b$ be the lecture with the larger index). Every lecture touching $b$ touches $a$, so $N(b) \setminus \{a\} \subseteq N(a) \cap N(b) = \emptyset$, i.e. $N(b) = \{a\}$. Thus $b$ determines $a$, and there are at most $n$ nested blocks.

*Crossing.* Neither arc contains the other, so in the cut picture $s_a < s_b < e_a < e_b$ (after naming). Exactly one of the two lectures, here $a$, has its *last cell* inside the other arc; call it the *left* member (this notion does not depend on where the circle was cut). Suppose $a$ is the left member of two different blocks $\{a,b\}$ and $\{a,b'\}$. The last cell $c$ of $a$ lies in $A_b$ and in $A_{b'}$. Then $b'$ overlaps $a$ and overlaps $b$ (both contain $c$), so $b' \in N(a) \cap N(b)$, contradicting that $\{a,b\}$ is a block. Hence every lecture is the left member of at most one crossing block, giving at most $n$ crossing blocks. □

# 3 The algorithm

**Idea.** By Lemma 1, lecture 1 hits exactly one region in any solution, i.e. it belongs to or touches exactly one block. We try every candidate block that hits lecture 1 as an *anchor*, rotate the circle so that the anchor's region starts at coordinate 0, unroll the circle to the line $[0,L]$, and decide with a left-to-right dynamic programme whether the remaining blocks can be chained so that $(\star)$ holds.

**Coordinates.** For an anchor region $R_0 = [0,\ell_0)$, lecture $j$ is placed at start $\sigma_j \in [0,L)$ (relative to the anchor) with end $\varepsilon_j = \sigma_j + d_j$. Besides $[\sigma_j,\varepsilon_j)$ we also use the copy $[\sigma_j - L, \varepsilon_j - L)$ that accounts for lectures crossing the anchor's start. For a position $p$ define

$$Q(p) = \max\big(\{\varepsilon_j : \sigma_j < p\} \cup \{\varepsilon_j - L : \text{all } j\}\big) \qquad \text{(furthest end of a lecture copy starting before } p\text{)},$$
$$T(p) = \min\{\varepsilon_j : \sigma_j \geq p\} \quad (\min\emptyset = \infty) \qquad \text{(earliest end of a lecture starting at or after } p\text{)}.$$

Both are computed for all $p$ in $O(n \log n)$ by sorting once and using a prefix maximum and a suffix minimum.

**Chains.** A *chain* is a sequence of blocks $X_0, \ldots, X_{k-1}$ ($X_0$ the anchor) whose regions are arcs $[\alpha_i, \beta_i)$ with

$$0 = \alpha_0 < \beta_0 \leq \alpha_1 < \beta_1 \leq \cdots \leq \alpha_{k-1} < \beta_{k-1} \leq L =: \alpha_k.$$

The *gap* $G_i = [\beta_i, \alpha_{i+1})$ (possibly empty) lies between $X_i$ and the next region; $G_{k-1}$ leads back to the anchor. A chain satisfies the *gap tests* if

$$T(\beta_i) > \alpha_{i+1} \quad \text{for all } i \in \{0,\ldots,k-1\} \quad \text{(no lecture lies strictly inside a gap)}, \tag{G1}$$
$$Q(\beta_i) \leq \alpha_{i+1} \quad \text{for all } i, \text{ if } k \geq 2 \quad \text{(no lecture meets two consecutive regions)}. \tag{G2}$$

For $k = 1$ only (G1) is needed.

**Lemma 3** (Chain lemma)**.** *For a chain as above, every lecture hits exactly one of the regions $R_i = [\alpha_i, \beta_i)$ if and only if (G1) and (G2) hold.*

*Proof.* The cells $[0,L)$ are the disjoint union $R_0 \cup G_0 \cup R_1 \cup \cdots \cup R_{k-1} \cup G_{k-1}$.

*No lecture hits no region $\iff$ (G1).* A lecture hits no region iff its arc is contained in the union of the gaps. An arc is contiguous and the gaps are separated by regions, so it lies inside a single gap $G_i$ (the last gap is followed, cyclically, by cell $0 \in R_0$). Then it does not wrap, so it has $\sigma_j \geq \beta_i$ and $\varepsilon_j \leq \alpha_{i+1}$, whence $T(\beta_i) \leq \alpha_{i+1}$. Conversely, if $T(\beta_i) \leq \alpha_{i+1}$ then the lecture attaining the minimum satisfies $\sigma_j \geq \beta_i$ and $\varepsilon_j \leq \alpha_{i+1}$, i.e. lies inside $G_i$.

*No lecture hits two regions $\iff$ (G2) (for $k \geq 2$).* If a copy of a lecture satisfies $\sigma < \beta_i$ and $\varepsilon > \alpha_{i+1}$, it contains the cells $\beta_i - 1 \in R_i$ and $\alpha_{i+1}$, which lies in $R_{i+1}$ (or equals $L \equiv 0 \in R_0$ when $i = k-1$); for $k \geq 2$ these are different regions. Such a copy exists iff $Q(\beta_i) > \alpha_{i+1}$. Conversely, suppose a lecture contains a

**Algorithm 1** SelectLectures(lectures, $L$)

build bit-masks of covered minutes and adjacency bit-sets $N(\cdot)$
$\mathscr{B} \leftarrow \{\{a,b\} : a \cap b \neq \emptyset,\ N(a) \cap N(b) = \emptyset\}$
**if** some block of $\mathscr{B}$ covers the whole circle **then**
    **return** it ▷ Corollary 1
**for all** $X_0 \in \mathscr{B}$ whose region hits lecture 1 **do** ▷ anchors
    rotate so that $X_0 = [0, \ell_0)$; compute $Q, T$ by prefix max / suffix min
    $C \leftarrow$ blocks lying in $[\ell_0, L]$, sorted by start
    **if** $L < T(\ell_0)$ **then**
        **return** $X_0$ ▷ $k = 1$
    **for all** $Y \in C$ in order of start **do**
        reach$[Y] \leftarrow \exists X \in \{X_0\} \cup C$ before $Y$ with reach$[X]$, $\beta_X \le \alpha_Y$, $Q(\beta_X) \le \alpha_Y < T(\beta_X)$
        **if** reach$[Y]$ **and** $Q(\beta_Y) \le L < T(\beta_Y)$ **then**
            **return** the chain ending in $Y$
**return** None

cell $x \in R_u$ and a cell $y \in R_v$ with $u \neq v$. Its arc contains one of the two cyclic paths from $x$ to $y$; say (after exchanging the names of $x, y$) the path that moves forward from $x$. Leaving $R_u$ the path passes through $\beta_u - 1$ and reaches the first cell $\alpha_{u+1}$ of the next region before reaching $y$. So the arc contains all cells from $\beta_u - 1$ to $\alpha_{u+1}$, and the copy of the lecture that does so has $\sigma < \beta_u$ and $\varepsilon > \alpha_{u+1}$: (G2) fails for $i = u$.

Hitting exactly one region means hitting neither none nor two or more; this is (G1)∧(G2). □

**Dynamic programme.** Let $C$ be the list of candidate blocks whose region lies in $[\ell_0, L]$ (these are exactly the blocks whose region is disjoint from $R_0$), sorted by start $\alpha$. Write reach$[Y]$ for "there is a chain $X_0, \ldots, Y$ satisfying (G1), (G2) between all consecutive blocks". Then

$$\text{reach}[X_0] = \text{true}, \qquad \text{reach}[Y] = \exists X \text{ before } Y :\ \text{reach}[X] \wedge \beta_X \le \alpha_Y \wedge Q(\beta_X) \le \alpha_Y < T(\beta_X),$$

and the chain closes at $Y$ if $Q(\beta_Y) \le L < T(\beta_Y)$. The case $k = 1$ is the test $L < T(\ell_0)$.

**Worked example (test T4).** Lectures (times in hours): L1 0–2, L2 1–3, L3 0:30–1, L4 8–10, L5 9–11, L6 8:30–9, L7 16–18, L8 17–19. The first anchor meeting lecture 1 is $X_0 = \{L1, L2\}$ with region $[0, 180)$ minutes (00:00–03:00). The other blocks inside $[180, 1440]$ include $Y_1 = \{L4, L6\}$ with region $[480, 600)$ and $Y_2 = \{L7, L8\}$ with region $[960, 1140)$. The tests are:

| Transition | values | $Q(\beta) \le \alpha$ | $\alpha < T(\beta)$ | result |
|---|---|---|---|---|
| $X_0 \to Y_1$ | $Q(180) = 180,\ \alpha = 480,\ T(180) = 540$ | ✓ | ✓ | ok |
| $Y_1 \to Y_2$ | $Q(600) = 660,\ \alpha = 960,\ T(600) = 1080$ | ✓ | ✓ | ok |
| $Y_2 \to X_0{+}L$ | $Q(1140) = 1140,\ \alpha = 1440,\ T(1140) = \infty$ | ✓ | ✓ | closes |

Hence the chain $X_0, Y_1, Y_2$ is accepted and $S = \{1, 2, 4, 6, 7, 8\}$ is returned, exactly the output of the program (Appendix A). Figure 1 shows the instance: the selected lectures are dark, the shaded bands are the three regions.

# 4 Correctness

**Theorem 1.** *SelectLectures returns a valid subset if one exists and None otherwise.*

*Proof. Soundness.* Suppose a set is returned. If it comes from the whole-circle case it is valid by Corollary 1. Otherwise it is the union of the blocks of a chain. The filter "region inside $[\ell_0, L]$" together with the condition $\beta_X \le \alpha_Y$ makes the regions pairwise disjoint and ordered, so the chain is as in Section 3; the accepted

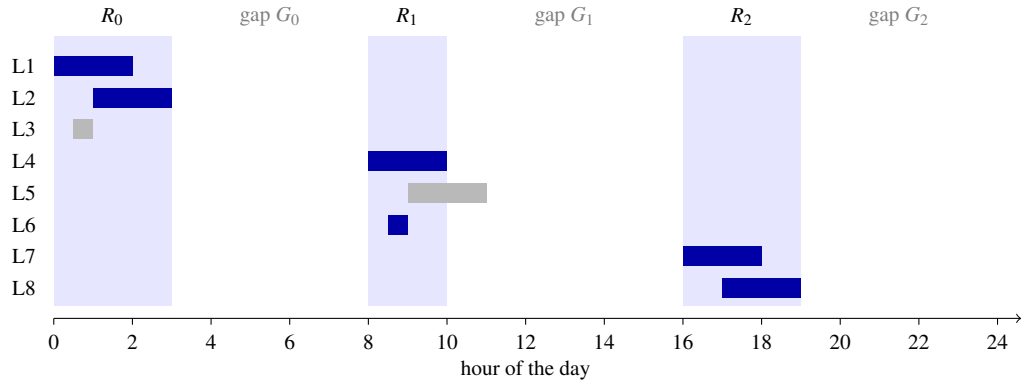



Figure 1: Test T4. Dark bars: selected lectures $\{1,2,4,6,7,8\}$; grey bars: lectures that are not selected (L3, L5). Shaded bands: regions of the chain $X_0, Y_1, Y_2$.

transitions and the closing test are exactly (G1) and (G2) for all gaps (for $k = 1$: (G1) only). By Lemma 3 every lecture hits exactly one region, and by Lemma 1 ($\Leftarrow$) the union of the blocks is valid.

*Completeness.* Let $S$ be valid. If $S$ contains a block whose region is the whole circle, then $k = 1$ (any other region would be hit twice) and the special case returns a valid block. Otherwise let $B_1, \ldots, B_k$ be the blocks of $S$ from Lemma 1; their regions are pairwise disjoint arcs, and lecture 1 hits exactly one of them, say $R_1$. Block $B_1$ is therefore among the anchors tried. With $B_1$ rotated to coordinate 0, every other region is an arc disjoint from $R_1 = [0, \ell_0)$, hence lies in $[\ell_0, L]$ and is in the list $C$; ordering them by start gives the cyclic order, i.e. a chain $X_0 = B_1, X_1, \ldots, X_{k-1}$. By $(\star)$ and Lemma 3, (G1) and (G2) hold along it. If $k = 1$ the test $L < T(\ell_0)$ succeeds. If $k \geq 2$, induction along the chain shows reach$[X_i]$ becomes true (the true predecessor $X_{i-1}$ passes the transition test, although the DP may record a different valid predecessor), and the closing test succeeds at $X_{k-1}$ (or at an earlier block, which is again valid by soundness). So some anchor returns a chain, and the algorithm does not output None. □

## 5 Complexity

Let $w$ be the machine word size (the program uses Python integers as bit-sets).

- *Masks and adjacency.* $n$ masks of $L$ bits and $\binom{n}{2}$ intersections cost $O(n^2 L/w)$. For the fixed $L = 1440$ this is a constant factor of about $1440/64 \approx 23$ word operations per pair.
- *Blocks.* Each of the $O(n^2)$ pairs needs one $n$-bit intersection $N(a) \cap N(b)$, i.e. $O(n^3/w)$ in total.
- *Number of blocks.* If a block covers the whole circle we stop. Otherwise there are $m \leq 2n$ blocks by Lemma 2, hence at most $2n$ anchors.
- *One anchor.* Sorting and computing $Q, T$ costs $O(n \log n)$; the DP examines all pairs of blocks, $O(m^2) = O(n^2)$.

Altogether

$$T(n) = O\big(n^2 L/w + n^3\big) = O(n^3) \text{ for fixed } L, \qquad S(n) = O(n^2 + nL) \text{ bits} = O(n^2) \text{ bits for fixed } L.$$

(The $nL$ term is linear in $n$, not constant; for $L = 1440$ it is dominated by $n^2$ as soon as $n > 1440$.) This is polynomial, compared with $\Theta(2^n)$ subsets for exhaustive search. The cubic bound is an upper bound; Section 7 measures the actual growth.

## 6 Implementation and reproducibility

**Source code.** The complete Python implementation, the sample input and a README are in the public repository

`https://github.com/Revanth422/DAA`

(the solver itself: `https://github.com/Revanth422/DAA/blob/main/lecture_solver.py`). The repository contains `lecture_solver.py` (algorithm, independent verifier, brute-force checker, test generators, benchmark), `sample.txt` and `mydata.txt` (example inputs), `README.md`, this report's LaTeX source `report.tex` with its generated files, and a GitHub Actions workflow that runs the tests on every push. The listing in Appendix B is read directly from `lecture_solver.py` when this document is built, so it is always the file that produced the results.

**Input format.** The first line contains $n$; then $n$ lines `HH:MM HH:MM` (start, end; 24-hour clock, `24:00` means midnight). Text after # is a comment. Plain integers are read as minutes after midnight. `mydata.txt` is simply an example of such a file for the reader's own data:

```
4
23:00 01:00
00:30 02:00
13:00 15:00
14:00 16:00
```

**Running.**

```
python3 lecture_solver.py sample.txt    solve an input file
python3 lecture_solver.py -demo         the 8 built-in tests
python3 lecture_solver.py -stress 1000  compare with brute force
python3 lecture_solver.py -bench        running times
pdflatex -shell-escape report.tex       rebuild this PDF (runs the code)
```

**Independent verification.** The solver works on a bit-mask graph. The checker `verify` and the exhaustive search `brute_force` do *not* reuse it: they decide overlap with the arithmetic test "arc $A$ contains the start cell of arc $B$, or vice versa" (two cyclic arcs intersect iff one contains the other's first cell). A wrong overlap model in the solver would therefore be detected. The stress tests also check that the bit-mask graph and the arithmetic graph are identical.

# 7 Experiments

## 7.1 Built-in tests and cross-checks

The following table is produced by the program while this document compiles. Each row runs the solver, the exhaustive search and the independent verifier.

| Test | Lectures (start–end) | Algorithm output | OK |
|---|---|---|---|
| T1 two overlapping lectures | L1 09:00–10:00, L2 09:30–10:30 | $S = \{1,2\}$ | ✓ |
| T2 night lecture wraps midnight | L1 23:00–01:00, L2 00:30–02:00 | $S = \{1,2\}$ | ✓ |
| T3 two chains around the day | L1 01:00–03:00, L2 02:00–04:00, L3 13:00–15:00, L4 14:00–16:00 | $S = \{1,2,3,4\}$ | ✓ |
| T4 four blocks, private lectures | L1 00:00–02:00, L2 01:00–03:00, L3 00:30–01:00, L4 08:00–10:00, L5 09:00–11:00, L6 08:30–09:00, L7 16:00–18:00, L8 17:00–19:00 | $S = \{1,2,4,6,7,8\}$ | ✓ |
| T5 lecture touched by two blocks | L1 01:00–03:00, L2 02:00–04:00, L3 05:00–07:00, L4 06:00–08:00, L5 03:30–05:30 | none | ✓ |
| T6 full-day lecture + private partner | L1 00:00–00:00, L2 10:00–11:00 | $S = \{1,2\}$ | ✓ |

| Test | Lectures (start–end) | Algorithm output | OK |
|---|---|---|---|
| T7 isolated lecture | L1 09:00–10:00, L2 09:30–10:30, L3 15:00–16:00 | none | ✓ |
| T8 touching endpoints do not overlap | L1 09:00–10:00, L2 10:00–11:00 | none | ✓ |

| Instance family | Instances | Solvable | Disagreements |
|---|---|---|---|
| random, period 24 | 200 | 35 | 0 |
| planted, period 24 | 200 | 200 | 0 |
| mutated, period 24 | 200 | 92 | 0 |
| planted, period 1440 | 200 | 200 | 0 |
| mutated, period 1440 | 200 | 88 | 0 |

**All built-in tests and all cross-checks passed.**

The tests cover wrap-around at midnight (T2), several blocks with private lectures (T4), a lecture touched by two blocks (T5), a full-day lecture (T6), an isolated lecture (T7) and lectures that merely touch at an endpoint (T8). The random families compare the polynomial algorithm with exhaustive search (up to 10 lectures). *Planted* instances are solvable by construction (random blocks, crossing or nested, with private lectures, randomly rotated and shuffled), so they test the "yes" answers; *mutated* instances change one endpoint, add or delete a lecture of a planted instance, and are close to the boundary between "yes" and "no". All random instances use fixed seeds and are therefore reproducible.

### 7.2 Running time

We measure `solve` for $L = 1440$ on two families. *Solvable*: planted instances. *Worst case*: lecture 1 is a half-day arc containing $n/2$ tiny private lectures, so about $n/2$ blocks meet lecture 1 (many anchors); the other half-day holds about $n/4$ blocks and one isolated lecture makes the instance unsolvable, so the DP runs for every anchor.

| $n$ | solvable: time (s) | $n$ | worst case: time (s) | growth |
|---|---|---|---|---|
| 100 | 0.001 | 100 | 0.005 | – |
| 200 | 0.003 | 200 | 0.025 | $\times$4.7 |
| 400 | 0.013 | 400 | 0.129 | $\times$5.2 |
| 800 | 0.047 | 800 | 0.716 | $\times$5.5 |

Doubling $n$ multiplies the time by a factor of roughly 4 to 7 on the adversarial family, between the factors 4 (quadratic) and 8 (cubic). The $O(n^3)$ bound is thus an upper bound which the experiment does not contradict; whether it is tight is left open. Times are machine dependent and were measured on the machine that built this PDF.

## 8 Conclusion and limitations

Every solution is a disjoint union of blocks, only $O(n)$ blocks can be candidates, and anchoring one block turns the circular problem into a linear dynamic programme. The algorithm runs in $O(n^3)$ time, is proved correct and agrees with exhaustive search on all tested instances. Limitations: time is discretised to whole minutes (the model with real-valued endpoints is analogous but would need coordinate compression); the cubic bound may not be tight; and the random cross-checks are limited by the exponential cost of the reference solver to small $n$. A natural improvement is to replace the $L$-bit masks by $O(1)$ arithmetic overlap tests.

## References


[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.

[2] E. J. Cockayne, R. M. Dawes, S. T. Hedetniemi, Total domination in graphs, *Networks* 10 (1980), 211–219.

[3] H. Gavlas, K. Schultz, Efficient open domination, *Electronic Notes in Discrete Mathematics* 11 (2002).

[4] M. C. Golumbic, *Algorithmic Graph Theory and Perfect Graphs*, 2nd ed., Elsevier, 2004.


## A Raw program output

Output of `python3 lecture_solver.py -demo` (the eight built-in tests):

```
--- T1 two overlapping lectures (n = 2) ---
Selected lectures (1-indexed): [1, 2]
lecture   1 09:00-10:00 supervised by [2]
lecture   2 09:30-10:30 supervised by [1]
Independent verification: PASSED
--- T2 night lecture wraps midnight (n = 2) ---
Selected lectures (1-indexed): [1, 2]
lecture   1 23:00-01:00 supervised by [2]
lecture   2 00:30-02:00 supervised by [1]
Independent verification: PASSED
--- T3 two chains around the day (n = 4) ---
Selected lectures (1-indexed): [1, 2, 3, 4]
lecture   1 01:00-03:00 supervised by [2]
lecture   2 02:00-04:00 supervised by [1]
lecture   3 13:00-15:00 supervised by [4]
lecture   4 14:00-16:00 supervised by [3]
Independent verification: PASSED
--- T4 four blocks, private lectures (n = 8) ---
Selected lectures (1-indexed): [1, 2, 4, 6, 7, 8]
lecture   1 00:00-02:00 supervised by [2]
lecture   2 01:00-03:00 supervised by [1]
lecture   3 00:30-01:00 supervised by [1]
lecture   4 08:00-10:00 supervised by [6]
lecture   5 09:00-11:00 supervised by [4]
lecture   6 08:30-09:00 supervised by [4]
lecture   7 16:00-18:00 supervised by [8]
lecture   8 17:00-19:00 supervised by [7]
Independent verification: PASSED
--- T5 lecture touched by two blocks (n = 5) ---
No valid subset exists.
--- T6 full-day lecture + private partner (n = 2) ---
Selected lectures (1-indexed): [1, 2]
lecture   1 00:00-00:00 supervised by [2]
lecture   2 10:00-11:00 supervised by [1]
Independent verification: PASSED
--- T7 isolated lecture (n = 3) ---
No valid subset exists.
--- T8 touching endpoints do not overlap (n = 2) ---
No valid subset exists.
All demo cases passed.
```

Output of `python3 lecture_solver.py sample.txt`:

```
--- sample.txt (n = 8) ---
Selected lectures (1-indexed): [1, 2, 4, 6, 7, 8]
lecture   1 00:00-02:00 supervised by [2]
lecture   2 01:00-03:00 supervised by [1]
```

```
lecture   3 00:30-01:00 supervised by [1]
lecture   4 08:00-10:00 supervised by [6]
lecture   5 09:00-11:00 supervised by [4]
lecture   6 08:30-09:00 supervised by [4]
lecture   7 16:00-18:00 supervised by [8]
lecture   8 17:00-19:00 supervised by [7]
Independent verification: PASSED
```

# B Source code

The complete file `lecture_solver.py`:

```python
#!/usr/bin/env python3
"""
Lecture Supervision -- Efficient Open Domination on a Circular-Arc Graph
=========================================================================

Problem
-------
n lectures repeat every day (period L = 1440 minutes). A lecture is a
circular arc [start, end) and may wrap past midnight. Choose a subset S such
that EVERY lecture overlaps EXACTLY ONE lecture of S (a lecture never counts
itself). Equivalently: S is an *efficient open dominating set* (total perfect
code) of the circular-arc "overlap" graph.

Overlap convention
------------------
Lectures are half-open arcs [start, end): a lecture ending at 13:00 and one
starting at 13:00 do NOT overlap. start == end means a full 24-hour lecture.

Usage
-----
python3 lecture_solver.py                 # type the lectures in interactively
python3 lecture_solver.py input.txt       # solve a file ("-" = read stdin)
python3 lecture_solver.py --demo          # the 8 built-in test cases
python3 lecture_solver.py --stress 500    # compare with brute force
python3 lecture_solver.py --bench         # running-time measurements
python3 lecture_solver.py --latex-report  # LaTeX fragment (used by report.tex)
python3 lecture_solver.py --latex-bench   # LaTeX timing table (used by report.tex)

Input format: first line n, then n lines "HH:MM HH:MM" (start end); "#" starts
a comment. Raw integers (minutes after midnight) are accepted as well.

Indexing: lectures are numbered 1..n in all output (0..n-1 inside the code).
"""
from __future__ import annotations

import argparse
import random
import sys
import time
from bisect import bisect_left

DAY = 1440  # minutes per day
INF = float("inf")


# ---------------------------------------------------------------------------
# 1. Parsing and basic geometry
# ---------------------------------------------------------------------------
def parse_time(token: str, period: int = DAY) -> int:
    """'HH:MM' (00:00..24:00) or a plain integer -> integer time in [0, period)."""
    token = token.strip()
    if ":" in token:
        parts = token.split(":")
        try:
            if len(parts) != 2:
                raise ValueError
            hh, mm = int(parts[0]), int(parts[1])
        except ValueError:
            raise ValueError(f"cannot read time {token!r}; use HH:MM") from None
```

```
        if not (0 <= mm <= 59 and 0 <= hh <= 24 and (hh < 24 or mm == 0)):
            raise ValueError(f"time out of range: {token!r}")
        return (hh * 60 + mm) % period
    try:
        return int(token) % period
    except ValueError:
        raise ValueError(f"cannot read time {token!r}; use HH:MM") from None


def fmt_time(t: int) -> str:
    return f"{t // 60:02d}:{t % 60:02d}"


def to_arcs(pairs, period=DAY):
    """[(start, end)] -> [(start, duration)], duration in 1..period."""
    arcs = []
    for s, e in pairs:
        s %= period
        d = (e - s) % period
        arcs.append((s, d if d else period))
    return arcs


def arc_mask(s: int, d: int, period: int) -> int:
    """Bit-mask of the unit cells covered by the arc (cyclic)."""
    full = (1 << period) - 1
    m = ((1 << d) - 1) << s
    return (m | (m >> period)) & full


def build_graph(arcs, period):
    """Return (cell masks, adjacency bitsets over lecture indices).

    This is the graph model used by the polynomial algorithm.
    """
    n = len(arcs)
    masks = [arc_mask(s, d, period) for s, d in arcs]
    adj = [0] * n
    for i in range(n):
        for j in range(i + 1, n):
            if masks[i] & masks[j]:
                adj[i] |= 1 << j
                adj[j] |= 1 << i
    return masks, adj


# ---------------------------------------------------------------------------
# 2. Independent checkers (verification only; they do NOT use build_graph)
# ---------------------------------------------------------------------------
def arcs_overlap(a, b, period) -> bool:
    """Do arcs a=(s1,d1) and b=(s2,d2) share a unit cell?

    Two cyclic arcs intersect iff one of them contains the start cell of
    the other. Pure modular arithmetic -- no bit-masks.
    """
    (s1, d1), (s2, d2) = a, b
    return (s2 - s1) % period < d1 or (s1 - s2) % period < d2


def adjacency_arithmetic(arcs, period):
    """Adjacency bitsets computed with arcs_overlap only."""
    n = len(arcs)
    adj = [0] * n
    for i in range(n):
        for j in range(i + 1, n):
            if arcs_overlap(arcs[i], arcs[j], period):
                adj[i] |= 1 << j
                adj[j] |= 1 << i
    return adj


def verify(pairs, chosen, period=DAY) -> bool:
    """True iff every lecture overlaps exactly one chosen lecture.

```

```
    Uses its own arithmetic overlap test, so it is independent of the
    bit-mask graph that the solver works on.
    """
    arcs = to_arcs(pairs, period)
    n = len(arcs)
    if not chosen or any(not (0 <= c < n) for c in chosen):
        return False
    adj = adjacency_arithmetic(arcs, period)
    cmask = 0
    for c in chosen:
        cmask |= 1 << c
    return all(bin(adj[v] & cmask).count("1") == 1 for v in range(n))


def brute_force(pairs, period=DAY):
    """Exponential reference solver (only for small n); independent of solve()."""
    n = len(pairs)
    adj = adjacency_arithmetic(to_arcs(pairs, period), period)
    for subset in range(1, 1 << n):
        if all(bin(adj[v] & subset).count("1") == 1 for v in range(n)):
            return [i for i in range(n) if subset >> i & 1]
    return None


# ---------------------------------------------------------------------------
# 3. The polynomial algorithm
# ---------------------------------------------------------------------------
def _run_start_len(umask: int, period: int):
    """Start cell and length of the (single) run of 1-cells in umask."""
    full = (1 << period) - 1
    if umask == full:
        return None
    rot = ((umask << 1) | (umask >> (period - 1))) & full  # cell c-1 -> bit c
    start_bit = umask & ~rot
    return start_bit.bit_length() - 1, bin(umask).count("1")


def candidate_blocks(arcs, masks, adj, period):
    """
    A *block* is an overlapping pair (a, b) with N(a) & N(b) = {} .
    Every solution is a disjoint union of blocks, so only these pairs matter.
    Returns list of (a, b, union_mask, run) ; run = None if union is the circle.
    """
    n = len(arcs)
    blocks = []
    for a in range(n):
        for b in range(a + 1, n):
            if adj[a] >> b & 1 and (adj[a] & adj[b]) == 0:
                u = masks[a] | masks[b]
                blocks.append((a, b, u, _run_start_len(u, period)))
    return blocks


def solve(pairs, period=DAY):
    """
    Return a sorted list of 0-based lecture indices forming a valid subset,
    or None if none exists. Time O(n^3) (plus O(n^2 L / w) for the masks).
    """
    n = len(pairs)
    if n < 2:
        return None
    L = period
    arcs = to_arcs(pairs, L)
    masks, adj = build_graph(arcs, L)
    blocks = candidate_blocks(arcs, masks, adj, L)

    # Special case: one block already wraps the whole circle.
    for a, b, _, run in blocks:
        if run is None:
            return [a, b]

    # Lecture 0 is touched by exactly one block in any solution -> anchor it.
    anchors = [k for k, blk in enumerate(blocks) if blk[2] & masks[0]]

```

```
    for x0 in anchors:
        chain = _try_anchor(x0, blocks, arcs, L)
        if chain is not None:
            return sorted(v for k in chain for v in blocks[k][:2])
    return None


def _try_anchor(x0, blocks, arcs, L):
    """Linear DP over blocks, with block x0 placed at coordinate 0."""
    s0, len0 = blocks[x0][3]

    # lecture arcs in coordinates where x0 starts at 0
    sig = [(s - s0) % L for s, _ in arcs]
    end = [sg + d for sg, (_, d) in zip(sig, arcs)]
    order = sorted(range(len(arcs)), key=lambda i: sig[i])
    sorted_sig = [sig[i] for i in order]
    pref_max = [-INF]                       # pref_max[k] = max end of first k
    for i in order:
        pref_max.append(max(pref_max[-1], end[i]))
    suf_min = [INF] * (len(arcs) + 1)       # suf_min[k] = min end from k on
    for k in range(len(arcs) - 1, -1, -1):
        suf_min[k] = min(suf_min[k + 1], end[order[k]])
    wrap_max = max(e - L for e in end)      # copies shifted by -L

    def Q(p):  # furthest right end of a lecture that starts before p
        return max(pref_max[bisect_left(sorted_sig, p)], wrap_max)

    def T(p):  # earliest end of a lecture that starts at or after p
        return suf_min[bisect_left(sorted_sig, p)]

    # nodes: (start, end, block-id); node 0 is the anchor itself
    nodes = [(0, len0, x0)]
    others = []
    for k, blk in enumerate(blocks):
        if k == x0:
            continue
        s_k, l_k = blk[3]
        ys = (s_k - s0) % L
        if ys >= len0 and ys + l_k <= L:   # disjoint from the anchor
            others.append((ys, ys + l_k, k))
    nodes += sorted(others)

    m = len(nodes)
    reach = [False] * m
    parent = [-1] * m
    reach[0] = True
    Qe = [Q(nd[1]) for nd in nodes]
    Te = [T(nd[1]) for nd in nodes]

    def chain_to(j):
        out = []
        while j != -1:
            out.append(nodes[j][2])
            j = parent[j]
        return out[::-1]

    # k = 1 : the anchor alone covers everything
    if L < Te[0]:
        return [x0]

    for j in range(1, m):
        ys = nodes[j][0]
        for i in range(j):
            if reach[i] and nodes[i][1] <= ys and Qe[i] <= ys < Te[i]:
                reach[j], parent[j] = True, i
                break
        # closing edge j -> anchor shifted by L (needs k >= 2)
        if reach[j] and Qe[j] <= L < Te[j]:
            return chain_to(j)
    return None


# ---------------------------------------------------------------------------
# 4. Input helpers
```

```
# ---------------------------------------------------------------------------
def parse_tokens(tok, period=DAY):
    """Token list [n, s1, e1, s2, e2, ...] -> list of (start, end)."""
    if not tok:
        raise ValueError("empty input")
    try:
        n = int(tok[0])
    except ValueError:
        raise ValueError(f"first value must be the number of lectures, got {tok[0]!r}") from
    None
    if n < 0 or len(tok) != 1 + 2 * n:
        raise ValueError(f"expected n and then {n} start/end pairs "
                         f"({1 + 2 * n} values), got {len(tok)} values")
    return [(parse_time(tok[1 + 2 * i], period), parse_time(tok[2 + 2 * i], period))
            for i in range(n)]


def read_file(path, period=DAY):
    """Read a file, or standard input when path is '-'. '#' starts a comment."""
    fh = sys.stdin if path == "-" else open(path)
    with fh:
        text = "\n".join(line.split("#")[0] for line in fh)
    return parse_tokens(text.split(), period)


def interactive(period=DAY):
    """Ask the user to type the lectures one by one."""
    n = int(input("Number of lectures n: "))
    pairs = []
    for i in range(1, n + 1):
        while True:
            try:
                s, e = input(f"Lecture {i} - start end (HH:MM HH:MM): ").split()
                pairs.append((parse_time(s, period), parse_time(e, period)))
                break
            except ValueError:
                print("  please enter two times such as 09:30 11:00")
    return pairs


def hm(s, e):
    return (parse_time(s), parse_time(e))


# ---------------------------------------------------------------------------
# 5. Built-in test cases (name, lectures, expected: solvable?)
# ---------------------------------------------------------------------------
TESTS = [
    ("T1 two overlapping lectures",
     [hm("09:00", "10:00"), hm("09:30", "10:30")], True),
    ("T2 night lecture wraps midnight",
     [hm("23:00", "01:00"), hm("00:30", "02:00")], True),
    ("T3 two chains around the day",
     [hm("01:00", "03:00"), hm("02:00", "04:00"),
      hm("13:00", "15:00"), hm("14:00", "16:00")], True),
    ("T4 four blocks, private lectures",
     [hm("00:00", "02:00"), hm("01:00", "03:00"), hm("00:30", "01:00"),
      hm("08:00", "10:00"), hm("09:00", "11:00"), hm("08:30", "09:00"),
      hm("16:00", "18:00"), hm("17:00", "19:00")], True),
    ("T5 lecture touched by two blocks",
     [hm("01:00", "03:00"), hm("02:00", "04:00"),
      hm("05:00", "07:00"), hm("06:00", "08:00"),
      hm("03:30", "05:30")], False),
    ("T6 full-day lecture + private partner",
     [hm("00:00", "00:00"), hm("10:00", "11:00")], True),
    ("T7 isolated lecture",
     [hm("09:00", "10:00"), hm("09:30", "10:30"), hm("15:00", "16:00")], False),
    ("T8 touching endpoints do not overlap",
     [hm("09:00", "10:00"), hm("10:00", "11:00")], False),
]


# ---------------------------------------------------------------------------
# 6. Random, planted and mutated instances (for testing)
```

```python
# ---------------------------------------------------------------------------
def random_instance(rng, n, period):
    return [(rng.randrange(period), rng.randrange(period)) for _ in range(n)]


def planted_instance(rng, period, k, extras):
    """Instance that is solvable BY CONSTRUCTION.

    The circle is cut into k windows (each >= 3 cells). Each window holds one
    block {a, b} (crossing or nested) and no lecture leaves its region, so
    every lecture overlaps exactly one region. Up to `extras` private lectures
    are added inside a \\ b or b \\ a. Returns a shuffled, rotated list.
    """
    if 3 * k > period:
        raise ValueError("too many blocks for this period")
    spare = period - 3 * k
    cuts = sorted(rng.randrange(spare + 1) for _ in range(k - 1))
    pieces = [b - a for a, b in zip([0] + cuts, cuts + [spare])]
    lectures, privates = [], []      # arcs as (start, length)
    pos = 0
    for piece in pieces:
        width = 3 + piece
        ell = rng.randint(3, width)
        rs = pos + rng.randint(0, width - ell)
        if rng.random() < 0.6:       # crossing pair
            q = rng.randint(1, ell - 2)
            p = rng.randint(q + 1, ell - 1)
            lectures.append((rs, p))
            lectures.append((rs + q, ell - q))
            privates.append((rs, q))             # inside a \ b
            privates.append((rs + p, ell - p))   # inside b \ a
        else:                        # nested pair b inside a
            v = rng.randint(1, ell)
            u = rng.randint(0, ell - v)
            lectures.append((rs, ell))
            lectures.append((rs + u, v))
            if u > 0:
                privates.append((rs, u))
            if u + v < ell:
                privates.append((rs + u + v, ell - u - v))
        pos += width
    for _ in range(extras):
        if not privates:
            break
        s, d = rng.choice(privates)
        x = rng.randint(0, d - 1)
        y = rng.randint(x + 1, d)
        lectures.append((s + x, y - x))
    shift = rng.randrange(period)
    pairs = [((s + shift) % period, (s + d + shift) % period) for s, d in lectures]
    rng.shuffle(pairs)
    return pairs


def mutate(rng, pairs, period):
    """Random small change: move an endpoint, add or drop a lecture."""
    pairs = list(pairs)
    kind = rng.randrange(3)
    if kind == 0 and pairs:
        i = rng.randrange(len(pairs))
        s, e = pairs[i]
        pairs[i] = (rng.randrange(period), e) if rng.random() < .5 else (s, rng.randrange(period
    ))
    elif kind == 1:
        pairs.append((rng.randrange(period), rng.randrange(period)))
    elif len(pairs) > 2:
        del pairs[rng.randrange(len(pairs))]
    return pairs


def _compare(inst, period):
    """Return (solvable, ok) comparing solve() with the brute force."""
    got = solve(inst, period)
    ref = brute_force(inst, period)
```

```
    ok = (got is None) == (ref is None)
    if got is not None:
        ok = ok and verify(inst, got, period)
    # the solver's bit-mask graph must agree with the arithmetic graph
    arcs = to_arcs(inst, period)
    ok = ok and build_graph(arcs, period)[1] == adjacency_arithmetic(arcs, period)
    return ref is not None, ok


def stress_category(kind, trials, seed, period, max_n):
    """Run one category; return (instances, solvable, disagreements)."""
    rng = random.Random(seed)
    solvable = bad = 0
    for _ in range(trials):
        if kind == "random":
            inst = random_instance(rng, rng.randint(2, max_n), period)
        else:
            k = rng.randint(1, min(max_n // 2, period // 3))
            inst = planted_instance(rng, period, k, rng.randint(0, max_n - 2 * k))
            if kind == "planted":
                if solve(inst, period) is None:    # must be solvable
                    bad += 1
            else:                                  # "mutated"
                inst = mutate(rng, inst, period)
        sol, ok = _compare(inst, period)
        solvable += sol
        bad += (not ok)
    return trials, solvable, bad


STRESS_PLAN = [  # (label, kind, period, max_n)
    ("random, period 24", "random", 24, 9),
    ("planted, period 24", "planted", 24, 10),
    ("mutated, period 24", "mutated", 24, 10),
    ("planted, period 1440", "planted", DAY, 10),
    ("mutated, period 1440", "mutated", DAY, 10),
]


def run_stress(trials_each, seed=1):
    return [(label,) + stress_category(kind, trials_each, seed + i, period, max_n)
            for i, (label, kind, period, max_n) in enumerate(STRESS_PLAN)]


# ---------------------------------------------------------------------------
# 7. Benchmark
# ---------------------------------------------------------------------------
def bench_instance(n, worst, seed=7):
    """Benchmark instance with about n lectures.

    worst=False: planted (solvable) instance.
    worst=True : adversarial for the number of anchors. Lecture 1 is a
        half-day arc that contains n/2 tiny private lectures, so every pair
        {lecture 1, tiny lecture} is a block meeting lecture 1 (n/2 anchors);
        the other half of the day holds ~n/4 crossing blocks, and one extra
        isolated lecture can never be supervised, so no solution exists and
        the DP has to run for every anchor.
    """
    if not worst:
        rng = random.Random(seed)
        k = max(1, n // 4)
        return planted_instance(rng, DAY, k, n - 2 * k)
    half = DAY // 2
    tiny = n // 2
    inst = [(0, half)]
    inst += [(2 * i, 2 * i + 1) for i in range(tiny)]
    kb = max(2, (n - tiny - 2) // 2)
    for i in range(kb):
        w = half + 4 * i
        inst += [(w, w + 2), (w + 1, w + 3)]
    inst.append((DAY - 3, DAY - 2))          # isolated lecture
    return inst
```

```
def _best_time(inst, repeat=3):
    best = INF
    for _ in range(repeat):
        t0 = time.perf_counter()
        solve(inst)
        best = min(best, time.perf_counter() - t0)
    return best


def run_bench(sizes=(100, 200, 400, 800)):
    rows = []
    for n in sizes:
        easy = bench_instance(n, False)
        hard = bench_instance(n, True)
        rows.append((len(easy), _best_time(easy), len(hard), _best_time(hard, 1)))
    return rows


# ---------------------------------------------------------------------------
# 8. Reporting
# ---------------------------------------------------------------------------
def report(pairs, name="input"):
    """Print the answer for one instance; return the chosen list or None."""
    sol = solve(pairs)
    print(f"--- {name} (n = {len(pairs)}) ---")
    if sol is None:
        print("No valid subset exists.")
        return None
    print("Selected lectures (1-indexed):", [c + 1 for c in sol])
    cmask = sum(1 << c for c in sol)
    arcs = to_arcs(pairs)
    adj = adjacency_arithmetic(arcs, DAY)
    for v, (s, d) in enumerate(arcs):
        sup = [u + 1 for u in range(len(arcs)) if (adj[v] >> u & 1) and (cmask >> u & 1)]
        print(f"lecture {v + 1:>3} {fmt_time(s)}-{fmt_time((s + d) % DAY)}"
              f" supervised by {sup}")
    print("Independent verification:", "PASSED" if verify(pairs, sol) else "FAILED")
    return sol


def _tex(s):
    return s.replace("&", r"\&").replace("_", r"\_").replace("#", r"\#")


def latex_report():
    out = [r"\begin{longtable}{p{4.6cm}p{5.6cm}p{2.6cm}c}",
           r"\toprule Test & Lectures (start--end) & Algorithm output & OK \\ \midrule \endhead"
    ]
    all_ok = True
    for name, pairs, expect in TESTS:
        got = solve(pairs)
        ref = brute_force(pairs)
        ok = ((got is None) == (ref is None) == (not expect)) and \
             (got is None or verify(pairs, got))
        all_ok &= ok
        lec = ", ".join(f"L{i + 1}\\,{fmt_time(s)}--{fmt_time(e)}"
                        for i, (s, e) in enumerate(pairs))
        res = "none" if got is None else "$S=\\{" + ",".join(str(c + 1) for c in got) + "\\}$"
        mark = "\\checkmark" if ok else "\\texttimes"
        out.append(f"{_tex(name)} & {{\\footnotesize {lec}}} & {res} & {mark} \\\\")
    out.append(r"\bottomrule \end{longtable}")

    rows = run_stress(200)
    out.append(r"\begin{center}\begin{tabular}{lrrr}")
    out.append(r"\toprule Instance family & Instances & Solvable & Disagreements \\ \midrule")
    total_bad = 0
    for label, trials, sol, bad in rows:
        total_bad += bad
        out.append(f"{_tex(label)} & {trials} & {sol} & {bad} \\\\")
    out.append(r"\bottomrule \end{tabular}\end{center}")
    good = all_ok and total_bad == 0
    out.append("\\par\\noindent\\textbf{" +
               ("All built-in tests and all cross-checks passed." if good else
                "WARNING: a test or cross-check failed.") + "}")
```

```
    return "\n".join(out)


def latex_bench():
    out = [r"\begin{center}\begin{tabular}{rrrrr}",
           r"\toprule $n$ & solvable: time (s) & $n$ & worst case: time (s) & growth \\ \midrule
    "]
    prev = None
    for n1, t1, n2, t2 in run_bench():
        growth = "--" if prev is None or prev == 0 else f"$\\times{t2 / prev:.1f}$"
        out.append(f"{n1} & {t1:.3f} & {n2} & {t2:.3f} & {growth} \\\\")
        prev = t2
    out.append(r"\bottomrule \end{tabular}\end{center}")
    return "\n".join(out)


# ---------------------------------------------------------------------------
# 9. Command line
# ---------------------------------------------------------------------------
def _run_demo():
    all_ok = True
    for name, pairs, expect in TESTS:
        sol = report(pairs, name)
        all_ok &= (sol is not None) == expect and (sol is None or verify(pairs, sol))
    print("All demo cases passed." if all_ok else "SOME DEMO CASES FAILED.")
    return 0 if all_ok else 1


def _run_stress(trials):
    if trials < 1:
        print("--stress must be a positive integer.", file=sys.stderr)
        return 2
    each = max(1, trials // len(STRESS_PLAN))
    bad_total = 0
    for label, n, sol, bad in run_stress(each):
        print(f"{label:<24} {n:>5} instances, {sol:>4} solvable, {bad} disagreements")
        bad_total += bad
    print("Stress test PASSED." if bad_total == 0 else "Stress test FAILED.")
    return 0 if bad_total == 0 else 1


def _run_bench():
    print(f"{'n':>5} {'solvable (s)':>13} | {'n':>5} {'worst case (s)':>15} {'growth':>8}")
    prev = None
    for n1, t1, n2, t2 in run_bench():
        growth = "" if not prev else f"x{t2 / prev:.1f}"
        print(f"{n1:>5} {t1:>13.3f} | {n2:>5} {t2:>15.3f} {growth:>8}")
        prev = t2
    return 0


def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                 formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("file", nargs="?", help='input file ("-" = standard input)')
    ap.add_argument("--demo", action="store_true", help="run the 8 built-in tests")
    ap.add_argument("--stress", type=int, metavar="N",
                    help="compare with brute force on about N random/planted/mutated cases")
    ap.add_argument("--bench", action="store_true", help="print running times")
    ap.add_argument("--latex-report", action="store_true")
    ap.add_argument("--latex-bench", action="store_true")
    a = ap.parse_args()

    if a.latex_report:
        print(latex_report())
        return 0
    if a.latex_bench:
        print(latex_bench())
        return 0
    if a.bench:
        return _run_bench()
    if a.stress is not None:
        return _run_stress(a.stress)
    if a.demo:
```

```
        return _run_demo()
    try:
        pairs = read_file(a.file) if a.file else interactive()
    except (ValueError, OSError) as err:
        print(f"Input error: {err}", file=sys.stderr)
        return 2
    report(pairs, a.file or "typed input")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```